

Практична робота №5. Семантичний пошук у колекції документів

Мета: Побудувати пошук по базі знань магазину, яка вже не вміщується в запит до моделі: поділити документи на фрагменти, перетворити їх на вектори локальною моделлю ембедінгів, зберегти індекс на диску, за запитом повертати top-k найближчих фрагментів з оцінкою схожості й фільтрами за метаданими, порівняти семантичний пошук із пошуком за ключовими словами на власному наборі запитів із відомими відповідями.

Зміст

1. Теоретичні відомості
2. Завдання до виконання
3. Критерії приймання
4. Контрольні запитання
5. Рекомендована література

1. Теоретичні відомості

У ПР3 і ПР4 усі правила магазину вміщувалися в один файл на чотири кілобайти, і він ішов у кожен запит цілком. У ПР4 з'ясувалося, що навіть таке — стаття витрат: правила з'їдають частину бюджету токенів щоразу, а історія діалогу тисне з іншого боку. Справжня база знань — це десятки документів: правила, інструкції до кожного товару, відповіді на часті запитання, довідники, внутрішні регламенти. Відправляти її в модель цілком неможливо, та й не потрібно: на питання про заряджання навушників стосується один абзац однієї інструкції. Отже, перед тим як питати модель, треба **знайти** ті кілька фрагментів, які стосуються справи. У цій роботі — лише пошук, без мовної моделі; у ПР6 він стане частиною помічника.

Опора, щоб виконувати роботу свідомо.

Ембедінг як координати змісту. Ембедінг — це вектор із кількох сотень чисел, який модель зіставляє тексту так, що тексти про одне й те саме опиняються поруч, а про різне — далеко. «Як повернути гроші за товар» і «кошти повертаються на рахунок протягом 7 днів» не мають жодного спільного слова, але їхні вектори близькі, бо модель навчено саме на цьому: зближувати перефразування й віддаляти стороннє. Моделі речень (sentence embeddings) дають один вектор на весь текст — речення, абзац, фрагмент — зазвичай розмірністю від 384 до 1024. Багатомовні моделі кладуть тексти різними мовами в один спільний простір, тож англійський запит може знайти українську відповідь — наскільки добре, залежить від моделі та її розміру, і це перевіряють, а не припускають. У кожній моделі є **обмеження на довжину входу** (у популярних — 512 токенів): довший текст вона мовчки обрізає. І в деяких моделях запит і документ кодуються по-різному — з префіксами на кшталт `query:` і `passage:`, — бо коротке питання й довгий абзац-відповідь не мають бути схожими *дослівно*, вони мають бути *відповідними*. Що саме вимагає модель, написано в її картці; пропущений префікс не дасть помилки, лише гіршу видачу.

Схожість. Близькість векторів міряють косинусом кута між ними: 1 — той самий напрямок, 0 — нічого спільного. Якщо вектори нормалізовані до одиничної довжини, косинус — це просто скалярний добуток, а схожість запиту з усіма фрагментами одразу — один добуток матриці на вектор. Оцінка схожості **відносна, а не абсолютна**: у багатьох моделях усі значення тісняться в межах 0,7–0,9, і «0,82» нічого не каже саме по собі — має значення лише порядок у видачі та відстань між сусідами. Тому поріг «нижче за яке значення відповіді немає» не береться з голови: його дивляться на власних запитах, порівнюючи оцінки тих, де відповідь є, з тими, де її немає.

Фрагменти. Індексують не документи, а їхні фрагменти, і на це дві причини. Перша — обмеження на довжину входу моделі. Друга важливіша: один вектор на інструкцію з десятьма розділами — це «середнє» між заряджанням, з'єднанням, доглядом і скиданням; він однаково далекий від кожного конкретного питання. Фрагмент має бути достатньо малим, щоб бути про одне, і достатньо великим, щоб бути зрозумілим без сусідів. Ділити можна за структурою — розділами, абзацами — або вікном фіксованої довжини з **перекриттям**, щоб думка, розрізана межею, вціліла хоч в одному з фрагментів. Уривок «утримуйте кнопку 10 секунд» сам по собі не каже, про який пристрій ідеться, — тому в текст фрагмента часто додають назву документа й заголовок розділу. Кожен фрагмент несе **метадані**: з якого він файлу, який розділ, категорію, аудиторію, дату, статус — за ними фільтрують видачу й показують джерело.



Індекс. Індекс — це матриця векторів $N \times d$, самі фрагменти й назва моделі, якою вектори отримано. Для сотень і тисяч фрагментів пошук — це повний перебір: одна матрична операція за мілісекунди, і нічого складнішого не треба. Бібліотеки наближеного пошуку найближчих сусідів (HNSW, IVF та інші) потрібні на мільйонах векторів, і платять за швидкість точністю — вони можуть пропустити справжнього сусіда. Індекс будують окремою командою й **зберігають на диску**: перераховувати ембедінги колекції на кожен запуск — марнотратство, а на кожен запит — абсурд. Поруч із векторами обов'язково лежить назва моделі: вектори різних моделей непорівнянні, і індекс, збудований однією, з запитом, закодованим іншою, мовчки видаватиме сміття. Змінилися документи, модель або правило поділу — індекс перебудовують.

Тор-к, поріг, фільтри. Пошук повертає k найближчих фрагментів, і k — компроміс: замало — правильна відповідь не потрапить, забагато — потонуть у сусідах. Поріг схожості відсікає слабкі влучення, і саме він дозволяє сказати «в базі цього немає» замість того, щоб завжди щось

повертати. Фільтри за метаданими накладаються **до** ранжування — тоді top-k рахується серед дозволених фрагментів — або **після**: тоді з k найкращих може лишитися нуль. Кілька фрагментів одного документа у видачі — нормально, але іноді їх варто згорнути до одного. Фільтр за аудиторією — не зручність, а межа безпеки: внутрішня інструкція оператора не має зринати у відповіді клієнтові лише тому, що в ній є потрібні слова.

Пошук за ключовими словами. Класичний пошук буде обернений індекс: для кожного слова — список фрагментів, де воно є. Ранжує BM25: слово важить більше, якщо часто трапляється в цьому фрагменті (з насиченням), рідко — в колекції загалом, і якщо фрагмент короткий. Це точно, швидко й пояснювано; на артикулах, кодах помилок, назвах моделей такий пошук зазвичай перемагає — модель ембедінгів ніколи не бачила `OR-X2-BLK`. Але він знає лише буквальні збіги: «гроші назад» не знайде «повернення коштів», а без зведення слів до основи «повернути» і «повернення» — різні слова, і в українській з її відмінюванням це болить сильніше, ніж в англійській. Запит іншою мовою для нього безнадійний. Тому в реальних системах обидва способи часто поєднують — **гібридний пошук** — зливаючи дві видачі в одну; який спосіб потрібен цій колекції, з'ясовують не міркуванням, а вимірюванням.

Як оцінюють пошук. Пошук оцінюють на наборі запитів, для яких наперед відомо, де правильна відповідь. Для кожного запиту дивляться, на якій позиції видачі опинився очікуваний документ: частка запитів, де він перший, — `hit@1`, у першій трійці — `hit@3`. Це дешево, повторювано й показує, які саме запити провалюються, — а звідти вже видно, що міняти: розмір фрагмента, модель, префікси, поріг. Міняти за раз щось одне, інакше невідомо, що спрацювало.

2. Завдання до виконання

Побудуйте **пошук по базі знань** магазину «Сузір'я» з ПР3–ПР4. База знань — папка `docs/` з архіву: шістнадцять документів із блоком метаданих у кожному (назва, категорія, товар, аудиторія, дата редакції, статус). Серед них є інструкції до чотирьох товарів із дуже схожими розділами, дві редакції правил доставки — чинна й архівна — та внутрішня інструкція оператора, призначена персоналу.

Команда побудови індексу читає всі документи, ділить їх на фрагменти, рахує вектори й **зберігає індекс на диск**. Застосунок при старті читає готовий індекс; якщо його немає — стартує й пояснює, що робити.

Користувач вводить запит довільним текстом і отримує **top-k найближчих фрагментів** з оцінкою схожості, назвою документа, джерелом і метаданими. Поруч — результати **пошуку за ключовими словами** по тих самих фрагментах, у тому самому форматі. Користувач може задати кількість результатів, поріг схожості та **фільтри за метаданими**: категорію, товар, аудиторію, статус.

Модель ембедінгів, поділ на фрагменти, індекс і пошук за словами — кожне в **окремому модулі**; веб-рівень не знає ані моделі, ані формату індексу. Бекенд — FastAPI.

Спосіб реалізації — на ваш розсуд (самостійно або скеровуючи AI-асистента), але ви маєте **розуміти й уміти пояснити** кожне рішення.

Матеріали. В архіві до цієї роботи, папка `pr5`: `requirements.txt`, скрипт перевірки середовища, зразок налаштувань, колекція `docs/` з описом, скрипт побудови індексу, каркас застосунку і папка `compare/` для порівняння. Каркас дає структуру проєкту, робочий веб-рівень зі сторінкою

на два стовпці результатів і фільтрами та розбір блоку метаданих у файлах; поділ на фрагменти, модель, індекс і пошук за словами — заготовки. Скрипт перевірки завантажує модель і кодує три пробні речення — **запустіть його заздалегідь**: разом із залежностями й моделлю це кілька сотень мегабайт.

Де вести роботу. Розпакуйте архів у корінь власного репозиторію курсу — поруч із папками попередніх робіт. Ключ доступу до API в цій роботі не потрібен, якщо ембедінги рахуються локальною моделлю. Збудований індекс до репозиторію не потрапляє — він відтворюється командою.

Орієнтири.

- Модель — локальна, через `sentence-transformers`; відправна точка в `.env.example` — багатомовна модель на 384 вимірів, якій вистачає CPU. Картка моделі на Hugging Face каже, чи потрібні їй префікси для запиту й тексту та яка в неї максимальна довжина входу. Альтернатива — ембедінги через той самий OpenAI-сумісний API, що в ПРЗ.
- Індекс — `pumpry`: матриця векторів, косинусна схожість, повний перебір. Для цієї колекції цього достатньо; бібліотека наближеного пошуку — за бажанням, але ви маєте пояснити, що вона додає й що забирає.
- Зберігання — `pumpry` для векторів і JSON для фрагментів з метаданими; назва моделі — обов'язково.
- Пошук за словами — BM25 із пакета `rank_bm25` або власний; спосіб розбити текст на слова — ваш, і він має бути однаковим для запиту й фрагментів.
- Параметри — `top-k`, поріг, розмір фрагмента — значення конфігурації, а не числа в коді.
- Час пошуку показуйте на сторінці окремо для кожного способу; вирішіть, чи входить у нього кодування запиту.

Продумайте самостійно.

- За якими межами ділити документи: за розділами, за абзацами, вікном фіксованої довжини? Що станеться з таблицею сервісних центрів і зі списком кодів помилок за кожного варіанта?
- Чи додавати до тексту фрагмента назву документа й заголовок розділу? Як це змінює вектор і як — те, що бачить користувач?
- Що зберігати в індексі, крім векторів, щоб через місяць його можна було безпечно використати?
- Що робити з архівною редакцією правил: не індексувати, індексувати з фільтром за замовчуванням, показувати з позначкою? Хто ухвалює це рішення — користувач чи код?
- Внутрішня інструкція оператора потрапила у видачу клієнтові. Де в вашій системі має стояти запобіжник — у фільтрі, який вмикає користувач, чи там, куди користувач не дістає?
- Фільтр до ранжування чи після? Що станеться з `top-5`, якщо після ранжування лишиться один дозволений фрагмент?
- Звідки взяти значення порога? Що ви побачите, порівнявши оцінку найкращого результату для запиту «кредит на автомобіль» з оцінками запитів, на які відповідь є?
- Оцінка BM25 і косинусна схожість — у різних одиницях. Чи можна їх порівнювати? Що зробити, щоб злити дві видачі в одну?
- Що з вашого рішення доведеться змінити, коли в колекції буде не шістнадцять документів, а шістнадцять тисяч?

Порівняння. Складіть **12–15 запитів** з відомими відповідями — файл `docs/`, у якому вона є. Види запитів: точний артикул або код помилки; синонім, якого немає в документі; та сама основа в іншій формі слова; перефразування; запит іншою мовою; запит з описками; питання, на яке в колекції є чинна й застаріла відповіді; питання, відповідь на яке — у внутрішньому документі; запит із фільтром за товаром; довге питання із зайвими деталями; запит поза колекцією. Приклад набору — в архіві.

Прожене набір обома способами. Для кожного запиту фіксуйте позицію очікуваного документа у видачі, оцінку першого результату, що стоїть вище за очікуваний документ, час пошуку. Зведіть `hit@1` і `hit@3` по кожному способу. Потім змініть **одну** річ — розмір фрагмента, модель, префікси, розбиття на слова — і прожене набір ще раз. Висновки — у файл.

Заготовка для цього — папка `compare/` з описом і прикладом набору.

3. Критерії приймання

- Команда побудови індексу читає **всі** документи `docs/`, ділить їх на фрагменти й **зберігає індекс на диску** разом із назвою моделі; застосунок при старті читає індекс, а не будує його.
- Фрагменти мають **осмислені межі й розмір**, який ви можете обґрунтувати; у кожного є джерело й метадані документа.
- Запит кодується **тією самою моделлю** і тим самим способом, що й фрагменти; пошук повертає top-k з **оцінкою схожості**, упорядкованих за спаданням.
- **Фільтри за метаданими** працюють — щонайменше аудиторія, статус, категорія, товар; ви можете пояснити, до чи після ранжування вони застосовуються і чому.
- **Поріг схожості** задається конфігурацією; його значення ви вивели з власних запитів, а не взяли з голови.
- **Пошук за ключовими словами** працює по тих самих фрагментах, з тими самими фільтрами, у тому самому форматі результату.
- Сторінка показує **обидва способи поруч**: оцінку, назву документа, джерело, метадані, текст фрагмента, час пошуку.
- Застосунок **не падає**: стартує без індексу й каже, що робити; порожній запит, невідомий режим, фільтр за неіснуючим полем дають зрозумілу помилку, а не 500.
- Модель створюється **один раз**; веб-рівень не знає ані моделі, ані формату індексу; параметри пошуку — з конфігурації.
- Проведено **порівняння двох способів** на 12–15 запитах з очікуваними документами; зведено `hit@1` і `hit@3`; одну зміну перевірено окремо; висновки зафіксовано.
- Ви можете **пояснити** кожне ухвалене рішення.

4. Контрольні запитання

1. Що таке ембедінг і чому тексти без жодного спільного слова можуть мати близькі вектори? Чого модель ембедінгів не «знає»?
2. Навіщо документи ділять на фрагменти, якщо модель може закодувати й довгий текст? Що стається з вектором занадто великого фрагмента і з занадто малого?
3. Як ви ділили документи й що при цьому сталося з таблицею сервісних центрів? Що ви додали до тексту фрагмента, крім самого уривка, і навіщо?

4. Що означає оцінка 0,83 у вашому пошуку? Чому її не можна порівнювати з оцінкою іншої моделі — і з оцінкою BM25?
5. Звідки взявся ваш поріг схожості? Скільки правильних відповідей він відсікає й скільки запитів поза колекцією пропускає?
6. Що зберігається у вашому індексі і що станеться, якщо змінити модель, не перебудувавши його?
7. Фільтр за аудиторією у вас — до чи після ранжування? Що зміниться, якщо навпаки? Чи може користувач його вимкнути, і чи має?
8. На яких запитах пошук за ключовими словами переміг семантичний і чому? Що з цього впливає для застосунку, який відповідає клієнтам?
9. Що змінила ваша одна зміна в порівнянні? Як ви впевнилися, що це саме вона?
10. Що у вашому рішенні перестане працювати на мільйоні фрагментів і що доведеться замінити?

5. Рекомендована література

1. Huyen C. AI Engineering: Building Applications with Foundation Models. Sebastopol: O'Reilly Media, 2025. 496 p. ISBN 978-1-0981-6630-4.
2. Jurafsky D., Martin J. H. Speech and Language Processing. 3rd ed. (draft). Stanford University, 2025. URL: <https://web.stanford.edu/~jurafsky/slp3/> (дата звернення: 15.09.2026).
3. Reimers N., Gurevych I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks // Proceedings of EMNLP-IJCNLP 2019. arXiv:1908.10084. URL: <https://arxiv.org/abs/1908.10084> (дата звернення: 15.09.2026).
4. Wang L., Yang N., Huang X. et al. Multilingual E5 Text Embeddings: A Technical Report. arXiv:2402.05672. URL: <https://arxiv.org/abs/2402.05672> (дата звернення: 15.09.2026).
5. Robertson S., Zaragoza H. The Probabilistic Relevance Framework: BM25 and Beyond // Foundations and Trends in Information Retrieval. 2009. Vol. 3, No. 4. P. 333–389.
6. Sentence Transformers Documentation. URL: <https://sbnet.net/> (дата звернення: 15.09.2026).
7. Hugging Face. Model card: intfloat/multilingual-e5-small. URL: <https://huggingface.co/intfloat/multilingual-e5-small> (дата звернення: 15.09.2026).
8. rank-bm25: A collection of BM25 algorithms in Python. URL: https://github.com/dorianbrown/rank_bm25 (дата звернення: 15.09.2026).
9. FastAPI Documentation. URL: <https://fastapi.tiangolo.com/> (дата звернення: 01.09.2026).